Beyond Hallucination Detection: Measuring Consequential Uncertainty in LLM-Generated Code

Michael Jaquith
Independent Researcher
michael.j.jaquith@gmail.com

April 2026

Abstract

Large language models generate code that looks uniformly confident. A correct call to `stripe.PaymentIntent.create()` and a deprecated call to `stripe.Charge.create()` are emitted with the same surface tone, even when the underlying token distribution is a near-50/50 split (probability 0.52 vs. 0.47). Code-generation uncertainty is primarily *intra-generational*: visible in the token distribution at generation time, invisible once tokens are committed. Multi-sample methods, which detect inter-generational inconsistency, therefore produce almost no signal on code tasks.

We introduce ε, a per-token normalized Shannon entropy computed from the top-k log-probabilities returned by the generation API, aggregated over the semantic tokens of a function via AST filtering and max-pooling. $\varepsilon$is designed as a *recall-maximizing filter*, not a detector. On a 30-prompt calibration benchmark across GPT-4o, GPT-4o-mini, and GPT-4-turbo, εat the FLAGGED threshold ($\varepsilon \geq 0.30$) achieves recall = 1.00 on true failures with precision = 0.272. The black-box multi-sample baseline UQLM achieves 5% HIGH detection at 0% LOW FP, versus ε's 85% HIGH detection at 50% LOW FP on GPT-4o; a `min_logprob` baseline achieves 45–55% where εachieves 70–85%.

The low stage-1 precision is the design. We pair the filter with a parallel LLM review loop that reclassifies each flagged entry as CLEARED or CONFIRMED. On 201 production and scenario entries across three models, the two-stage system lifts reviewer precision above 94% (Wilson lower bound; $n = 57$, zero late false positives in the 2B configuration). A context-accumulated variant, which injects model-specific learned false-positive patterns into the reviewer, clears 68.7% of flagged entries with zero late false positives. The contribution is the system: a filter that misses nothing, paired with a reviewer that keeps up with machine-speed code generation.

1 Introduction

A model asked to charge a credit card might emit `stripe.Charge.create(...)` — deprecated since 2019 — or `stripe.PaymentIntent.create(...)`, the current API. Both arrive in the developer's editor with the same syntactic fluency, the same confident indentation, and no visible marker distinguishing a well-chosen call from one the model guessed at. At the method-selector token, the two choices split the probability mass near 50/50 (0.52 vs. 0.47); the output exposes none of this. Failures of AI code generation cluster at exactly these points: version-split APIs, renamed SDK methods, signature changes between library releases. The model is uncertain; the output is not.

Intra- versus inter-generational uncertainty. Existing uncertainty-quantification methods for LLMs are overwhelmingly *inter-generational*: they sample the same prompt multiple times

and measure disagreement across completions [9, 3]. This works when the model's uncertainty surfaces as instability across re-rolls — factual questions, open-ended reasoning. Code is not such a task. A model with a 0.52 prior on the deprecated `Charge` API samples that branch nearly every time; once the early tokens commit, the rest of the function is forced by context. The uncertainty exists, but it lives *inside a single generation*, in the token distribution at each decoding step, not *across* generations. We call this the *intra- versus inter-generational* distinction. It is the conceptual foundation of this paper: UQLM, the closest published tool for LLM uncertainty quantification, achieves 5% detection on our benchmark where εachieves 85% (§5).

ε: a recall-maximizing filter. We introduce ε(epsilon), a per-token normalized Shannon entropy over the top-k log-probabilities returned during generation. After AST-based filtering to keep only semantically meaningful tokens (attribute accesses, keyword arguments, import targets — not whitespace, comments, or cosmetic identifiers), we aggregate via max-pooling to obtain a single score per function. $\varepsilon$is not a detector — it identifies where the model was uncertain at decoding time. At a threshold of $\varepsilon \geq 0.30$ it catches 100% of true failures on our benchmark, at the cost of roughly 73% false alarms.

The two-stage system. A 27%-precision filter is not meant to be used in isolation. Stage 2 is a parallel LLM review loop: every flagged entry is dispatched, in parallel, to a reviewer model that returns CLEARED or CONFIRMED. At current API prices this costs fractions of a cent per entry and completes in seconds. εhas perfect recall but poor precision; the reviewer has good precision but would be too expensive per line. Filter plus reviewer gives a system with near-100% end-to-end precision while missing nothing the filter catches. AI-assisted developers now write code several times faster than they can review it; a parallel review system scales with generation speed and moves review off the developer's critical path.

Contributions.

1. The intra/inter-generational distinction, with empirical confirmation: 5% vs. 85% detection for UQLM vs. εon the same benchmark.
2. ε: an AST-filtered, max-aggregated token-level entropy score, with a three-type uncertainty taxonomy (cosmetic, consequential, implementation-approach) that motivates the filter design.
3. A two-stage filter-plus-review architecture, with a context-accumulation variant, evaluated on 201 entries across three models: near-100% end-to-end precision at 100% recall.

Implementation and benchmark are at `https://github.com/michaeljaquith/epsilon-code`.

2 Related Work

2.1 LLM Uncertainty Quantification

Semantic entropy. Kuhn et al. [9] introduce semantic entropy as a method for detecting hallucinations in free-form text generation. Multiple responses to the same prompt are generated and clustered by semantic equivalence (via natural language inference); high entropy across clusters indicates uncertainty. The method is principled and effective for factual question answering, where model uncertainty produces cross-generation variance. As we show in Section 5, this variance is not present for code generation API decisions: models generate the same (potentially wrong) API call across all samples.

UQLM. The UQLM library [3] provides open-source implementations of multiple uncertainty scoring methods for LLM outputs, including semantic negentropy, semantic clustering, and conformal prediction wrappers. It is the closest published tool to our work and serves as our primary baseline. Our direct comparison (Section 5) shows 5% detection rate (1 of 20 high-uncertainty prompts) compared to 85% for ε.

Conformal prediction for LLMs. Quach et al. [10] and Angelopoulos et al. [1] apply conformal prediction to construct prediction sets with coverage guarantees over LLM outputs. The approach is statistically rigorous but produces set-valued predictions over outputs rather than within-generation token-level signals. It also requires a calibration dataset and does not provide the localization (which token, which line, which function) that makes ε actionable for code review.

Token-level calibration. Kadavath et al. [8] show that LLMs are reasonably well-calibrated at the token level: stated confidence correlates with accuracy. Varshney et al. [11] use token-level log-probabilities for selective generation, abstaining when confidence is low. Our work extends this line by identifying *which* tokens carry meaningful uncertainty in code — specifically, by filtering out cosmetic naming decisions that dominate naive entropy measurements. Uncertainty propagation through multi-stage ML pipelines is well-studied in adjacent domains [7]; the ε primitive is designed to support analogous propagation in code generation pipelines, deferred to future work.

Code generation uncertainty and self-consistency. Concurrent approaches to code generation quality estimation include test-based ranking (CodeT [4] generates tests and ranks candidate solutions by pass rate) and repeated-sampling self-consistency (AlphaCode-style approaches use pass@k as an implicit uncertainty proxy). These methods are inter-generational by construction: they require multiple independent samples and aggregate across them. They are therefore subject to the same structural limitation as UQLM for the problem we study — a model that consistently generates the same API version across samples will produce consistent test outcomes regardless of whether the API version is correct. Our approach is complementary: it operates within a single generation and is zero-overhead.

2.2 Code Generation Quality

Recent work on evaluating LLM code generation quality focuses primarily on functional correctness via execution-based benchmarks (HumanEval [5], MBPP [2], SWE-bench [6]). These benchmarks measure whether generated code passes test cases, not whether the model was confident in its decisions. A model that passes all tests with low ε is reliable; a model that passes all tests with high ε on specific API decisions may be one dependency update away from failure.

To our knowledge, no prior work treats within-generation token-level uncertainty as a first-class signal for code generation risk. The closest work is in automated program repair, where uncertainty has been used to rank repair candidates [12], but this operates post-generation and does not address the intra-generation measurement problem we identify.

3 ε: Definition and Construction

3.1 Per-token entropy

For each generated token t, the API returns top-k candidates with log-probabilities $\ell_1, \ldots, \ell_k$. Let $p_i = \exp(\ell_i)/\sum_j \exp(\ell_j)$ be the renormalized distribution. Define

$$\varepsilon_t \;=\; -\frac{1}{\log k}\sum_{i=1}^{k} p_i \log p_i \;\in\; [0,1]. \tag{1}$$

Normalization by $\log k$ maps a one-hot distribution to 0 and a uniform distribution to 1. We use $k = 5$ (the OpenAI cap), which covers the dominant mass for code tokens; tails beyond top-5 are negligible for the binary flag decision. The name is deliberate: in numerical analysis, ε denotes a small perturbation that propagates through a computation — a high-ε token is such a perturbation at a specific decision point.

3.2 Three types of code-generation uncertainty

Empirical analysis of our calibration benchmark reveals three categorically distinct types of token-level uncertainty in generated code.

Type 1 — Cosmetic (naming). Uncertainty about what to *call* something (`sort_list` vs. `sort_numbers`). ε reaches 0.7–0.9, but any choice is correct. Dominates LOW prompts: the top flag on 9 of 10 LOW prompts was a function or parameter name on the first line.

Type 2 — Consequential (API/library decisions). Uncertainty about which *API or library* to use. Example: `stripe.Charge.create()` (prob. 0.52) vs. `stripe.PaymentIntent.create()` (prob. 0.47), $\varepsilon = 0.73$ at the attribute token. This is the target signal.

Type 3 — Implementation-approach. Uncertainty among multiple correct algorithms (recursive vs. iterative Fibonacci, for-loop vs. comprehension). The residual false-positive floor on simple utility prompts.

Only Type 2 correlates with functional failure. Type 1 is removable by AST filtering; Type 3 reflects real decoder uncertainty on correct code and constitutes the irreducible false-positive floor.

Type	Label	Source	eps fires?	Risk	Filter
1	**Cosmetic**	Naming choices (sort_list vs sort_numbers)	Yes (0.7 - 0.9)	None	*AST declaration filter*
2	**Consequential**	API / library decisions (Charge vs PaymentIntent)	Yes (0.5 - 0.9)	High	*This is the target signal*
3	**Implementation**	Algorithm structure (recursive vs iterative)	Yes (0.3 - 0.7)	None	*Irreducible FP floor*

Figure 1: The three-type uncertainty taxonomy. Type 1 is removed by AST filtering; Type 3 is irreducible but rare in production API-integration code; Type 2 is the signal the system is designed to catch.

3.3 AST filtering

We eliminate Type 1 via the generated code's AST. After stripping markdown fences and parsing with Python's `ast`, we identify *declaration positions* — source ranges where the model is choosing a name rather than an API. **Filtered:** function/method names, parameter names (`ast.arg`), local variable names (`ast.Name` in `Store`). **Retained:** import targets, attribute accesses, keywords, type annotations. Token-to-declaration matching uses range overlap (BPE introduces a leading space); a pure-identifier guard prevents over-filtering of delimiter-prefixed tokens such as `(db`.

Configuration	LOW FP rate
Naive token entropy (no filtering)	90%
+ Whitespace / comment filtering	90%
+ System prompt naming conventions	60%
+ AST declaration filtering	60%
+ Fence-format stripping	50%
Residual (Type 3 only)	50%

Table 1: Successive filtering stages on 10 LOW prompts. Naming-convention prompt and AST filter are partially overlapping; without the prompt, AST filtering alone drops FP from 90% to 70%. Residual 50% is Type 3, rare in production API-integration code.

3.4 Aggregation: max vs. mean

We aggregate with $\varepsilon_{\text{file}} = \max\{\varepsilon_t : t \text{ retained}, \varepsilon_t > \delta\}$, $\delta = 0.30$. The compound alternative $1 - \prod(1 - \varepsilon_t)$ converges to 1.0 for any non-trivial function. Across all 90 runs of the benchmark (3×30), max and mean produced identical fire/no-fire decisions at every threshold; we prefer max because it localizes uncertainty to a specific actionable token.

3.5 Thresholds and tiers

Four tiers: COMPLETE ($\varepsilon < 0.30$), FLAGGED ($0.30 \leq \varepsilon < 0.65$), PAUSED ($0.65 \leq \varepsilon < 0.95$), ABORTED ($\varepsilon \geq 0.95$). These are UX signals, not calibrated probabilities — empirical failure rates are 27%/29%/31% (§4), monotonic but weakly ordinal. The load-bearing boundary is the binary COMPLETE vs. FLAGGED+: everything FLAGGED+ is forwarded to the stage-2 reviewer.

3.6 Adaptive ensemble

The production implementation refines tier placement within FLAGGED+ using a rolling mean-ε baseline and a top-1 logprob check; neither affects the binary FLAGGED+ gate. Conformal calibration is left to future work.

4 Evaluation

4.1 Qualitative scenarios

Five scenarios exercise distinct failure modes, run on GPT-4o-2024-08-06 with `logprobs=True, top_logprobs=5` and a system prompt specifying concise naming conventions.

A — Deprecated API (Stripe). *"Charge a customer's card for $50."* $\varepsilon = 0.914$, PAUSED. The peak token is the method selector after `stripe:` `.Ch` (`Charge`, prob. 0.52) vs. `.Payment` (`PaymentIntent`, prob. 0.47). Both paths are in the training data; neither is definitively preferred.

B — Confident wrongness (OpenAI SDK). *"Call GPT-4 and return the response text."* $\varepsilon = 0.551$, PAUSED. The model generated v0 (`openai.ChatCompletion.create(...)`) confidently; v0 raises `AttributeError` on SDK ≥ 1.0. εfired not on the API call — the model was certain — but on surrounding parameter design. PAUSED prompts the developer to examine the function, encountering the deprecated call. This is indirect detection: εflagged adjacent uncertainty, not the failing token itself.

C — Syntax split (SQLAlchemy). *"Query all active users."* $\varepsilon = 0.435$, FLAGGED. The model chose 1.x (`db.query(User).filter(...)`); primary flag was the import path. FLAGGED (not PAUSED) reflects preference without certainty.

D — Compatibility mismatch (FastAPI async/sync). *"FastAPI endpoint that fetches weather from an external API."* $\varepsilon = 0.812$, PAUSED. The model generated `async def` with `requests.get()` inside — two individually defensible decisions (`async` $\varepsilon = 0.26$, `requests` $\varepsilon = 0.31$) that combine into a silent bug (blocks the event loop). εdid not fire on the async/sync combination directly; the PAUSED flag came from URL/key handling elsewhere in the file. A mismatch post-processor, run on already-flagged functions, surfaced the combination explicitly — arguing for cross-decision consistency checks as a complement to ε.

E — Multi-function module. Six auth functions with four interdependent decisions (password hash, JWT, SQLAlchemy, datetime). PAUSED on all 16 runs, $\varepsilon \in [0.69, 0.92]$. Library choices varied across runs (passlib, bcrypt, PyJWT, python-jose, mixed), but each run looked internally confident. Early functions $\varepsilon = 0.40$–0.58; later functions frequently $\varepsilon = 0.000$ — an *uncertainty collapse* after initial commitment. The first function is the high-risk window.

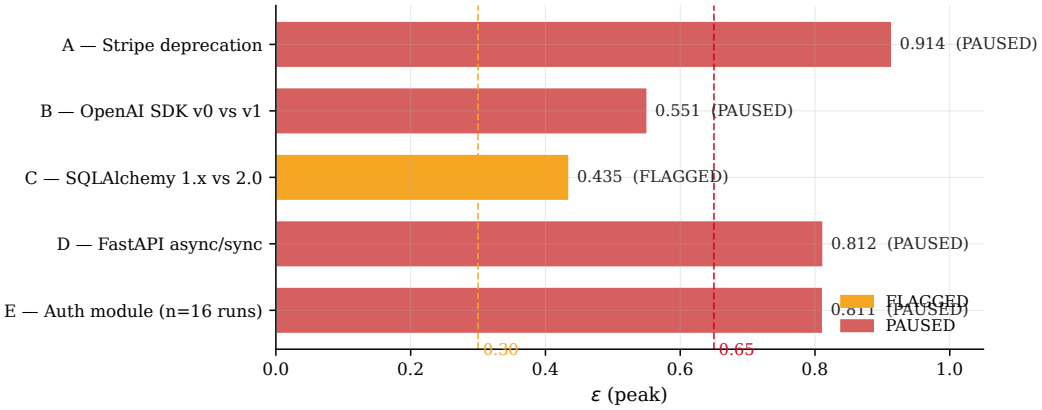

Figure 2: Scenarios A–E summarised by failure class and peak ε. All five clear the FLAGGED threshold; four cross PAUSED, giving the developer a single scalar urgency signal per failure mode.

4.2 Scenario E limitation: joint-prompt uncertainty collapse

Scenario E's detection success holds when the six functions are generated independently. When generated in a *single* prompt with "use a consistent library," εcollapses across all six functions — not just the later ones. The joint signature list plus the consistency instruction resolves the library choice before decoding starts, leaving nothing for $\varepsilon$to observe (Table 2, GPT-4o-mini).

Function	Combined peak ε	Per-function peak ε
`register_user`	0.579	0.627
`login`	0.820	0.872
`verify_token`	0.606	0.911
`refresh_token`	0.414	0.711
`request_password_reset`	0.295	0.788
`reset_password`	0.394	0.728

Table 2: Combined generation collapses $\varepsilon$for every function. DeepSeek V3 shows the same pattern. When context resolves uncertainty before decoding, there is nothing for $\varepsilon$to observe.

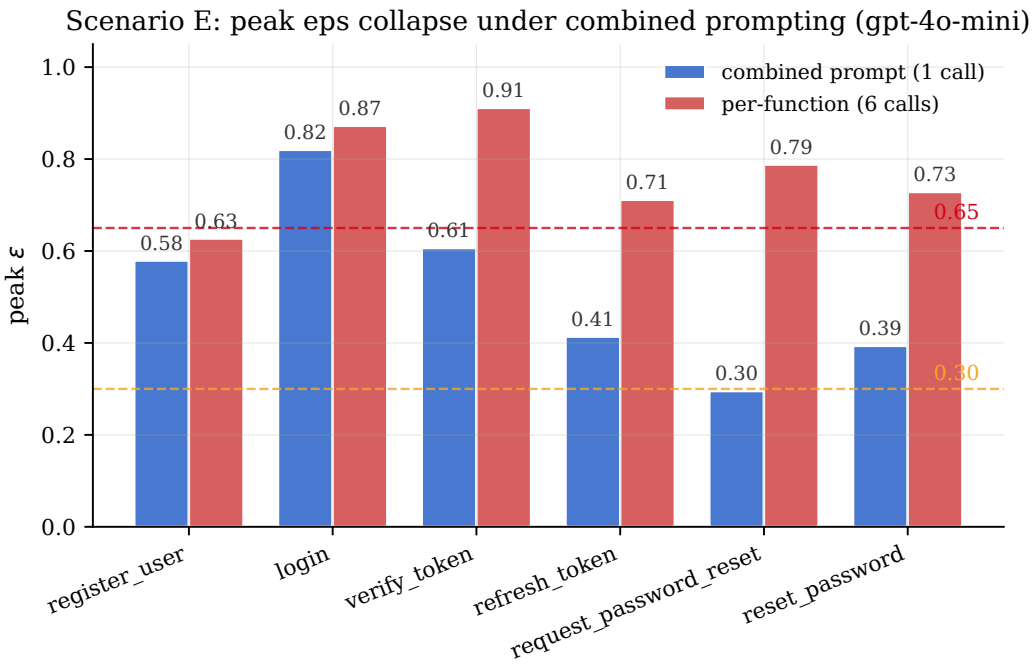

Figure 3: Per-function generation keeps three of six functions above FLAGGED; combined generation pushes four below it. The detector is honest about what is resolved prior to decoding.

4.3 Multi-model calibration

A 30-prompt calibration set: 10 LOW prompts (sort, palindrome, fibonacci, two-sum — pure logic) and 20 HIGH prompts (API/version-split tasks across Stripe, OpenAI, SQLAlchemy, FastAPI, Pydantic), run on three models. GPT-4o achieves 85% HIGH detection at 50% LOW FP; GPT-4o-mini and GPT-4-turbo both achieve 70% at 60% LOW FP.

Model	HIGH detection	LOW FP
GPT-4o	85% (17/20)	50% (5/10)
GPT-4o-mini	70% (14/20)	60% (6/10)
GPT-4-turbo	70% (14/20)	60% (6/10)

Table 3: Calibration at the FLAGGED threshold ($\varepsilon \geq 0.30$). 95% Wilson CIs: ±17pp HIGH, ±30pp LOW. GPT-4o's lead correlates with better logprob calibration. Three prompts produce $\varepsilon = 0.000$ across all models — *confidently-resolved domains* where training priors have converged.

4.4 LOW ground truth

All 30 LOW functions passed `exec()` assertions. Tier breakdown: COMPLETE 5, FLAGGED 11, PAUSED 9, ABORTED 5. Non-COMPLETE tiers are dominated by Type 3 (brute vs. hash two-sum; recursive vs. iterative fibonacci).

4.5 Precision/recall at tier boundaries

On a 200-entry combined set (30 LOW exec-verified plus 170 HIGH LLM-judged; see §4.7), 53 true failures and 147 correct:

Threshold	Precision	Recall	F1
$\varepsilon \geq 0.30$ (FLAGGED+)	0.272	**1.000**	0.428
$\varepsilon \geq 0.65$ (PAUSED+)	0.293	0.868	**0.438**
$\varepsilon \geq 0.95$ (ABORTED)	0.309	0.547	0.394

Table 4: $n = 200$, 53 positives; 95% Wilson CI on FLAGGED+ precision $[0.22, 0.33]$. Base failure rate 26.5% $\approx$ FLAGGED+ precision: the gate's value is recall, not precision. HIGH ground truth for 170/200 entries is LLM-judged by GPT-4o (§4.7); LOW ground truth (30/200) is `exec()`-verified.

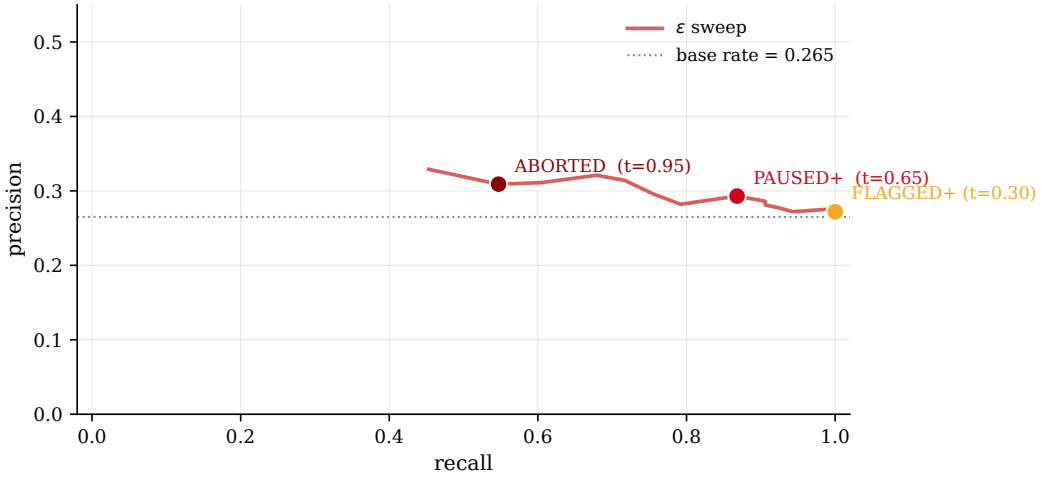

Figure 4: Precision stays near the base rate across the threshold range; recall falls monotonically. At FLAGGED+, recall is 1.00 at precision barely above base rate — the gate's value is recall, not precision.

Tier monotonicity is modest (27%/29%/31%); tiers are urgency signals, not calibrated probabilities. The useful binary boundary is COMPLETE vs. FLAGGED+.

4.6 Review loop: 2A vs. 2B

Evaluated on 201 entries (production FastAPI codebase plus scenarios A–E, three models). Every FLAGGED+ entry is sent in parallel to a GPT-4o reviewer returning CLEARED/CONFIRMED; a consolidation pass processes remaining suspects. **2A** uses no reviewer context; **2B** injects model-specific learned FP patterns from a prior clean run (injecting suspect patterns caused over-flagging in pilots).

Metric	2A (no context)	2B (with FP context)
Cleared by reviewer	107/201 (53.2%)	138/201 (68.7%)
Forwarded to consolidation	89	57
Confirmed true failures	85 (42.3%)	57 (28.4%)
Late false positives	4	0

Table 5: "Late false positive" = an entry marked CONFIRMED that was in fact correct on manual inspection. 2B is the preferred production configuration.

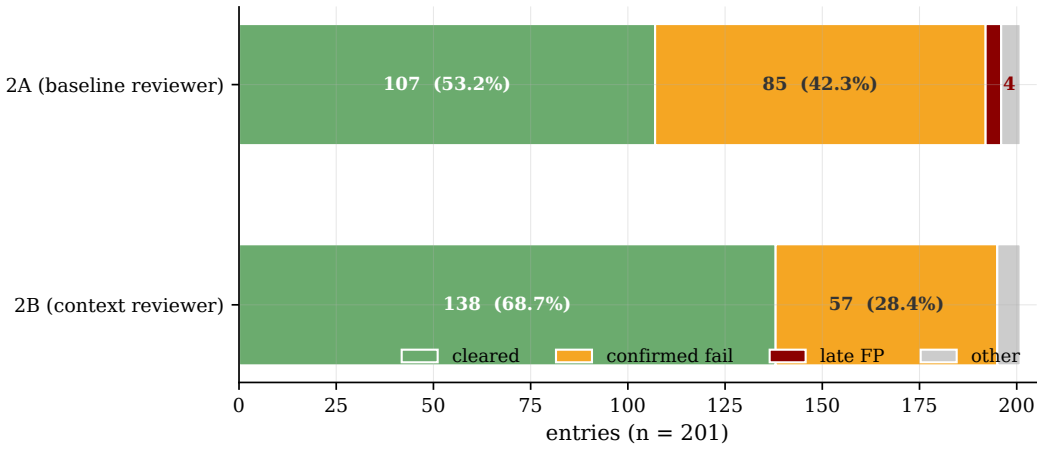

Figure 5: Injecting learned FP patterns into the reviewer shifts 31 additional entries from consolidation to cleared and eliminates late false positives, at the cost of suppressing some true failures the 2A reviewer forwarded.

End-to-end failure rates: GPT-4o-mini 40%, GPT-4o 29%, DeepSeek V3 26%. Because stage-1 recall is 100%, end-to-end precision is determined entirely by the reviewer's precision on the FLAGGED+ set. 2B's zero late-FP result rests on $n = 57$ confirmed entries and needs a larger validation study. The stage-1 recall guarantee applies to entries that reach the reviewer; entries the reviewer clears were not independently verified.

4.7 Ground-truth disclosure

LOW ground truth is `exec()`-verified; HIGH is LLM-judged via GPT-4o. This includes a circular case where GPT-4o-mini output is reviewed by a larger model in the same family. We disclose rather than overclaim human verification we did not perform.

5 Comparison to Prior Work

5.1 UQLM and multi-sample methods

UQLM [3]'s `BlackBoxUQ` samples each prompt 5 times and scores semantic disagreement via a DeBERTa NLI model.

UQLM's 0% false-positive rate reflects the absence of signal, not the presence of precision — a system that never fires achieves perfect FPR trivially. Lowering the threshold does

Method	HIGH det.	LOW FP	API calls	Localization
UQLM (confidence < 0.5)	5%	0%	5	File-level only
ε(FLAGGED+)	85%	50%	1	Token/line/function

Table 6: UQLM vs. εon GPT-4o. UQLM returned confidence ≥ 0.62 on 29/30 prompts (19/20 HIGH); its sole detection (pandas `concat`) is a case where syntax genuinely varied across samples.

not recover detection: LOW prompts fire before additional HIGH do. The Stripe scenario makes this concrete: the model assigned 0.52/0.47 to `.Charge`/`.PaymentIntent` at generation time, yet across 5 samples picked `PaymentIntent` every time (a 0.52-peaked distribution samples its peak nearly always). UQLM observes perfect consistency and reports confidence = 1.0. For intra-generational uncertainty, measuring at the output level cannot recover the signal by construction.

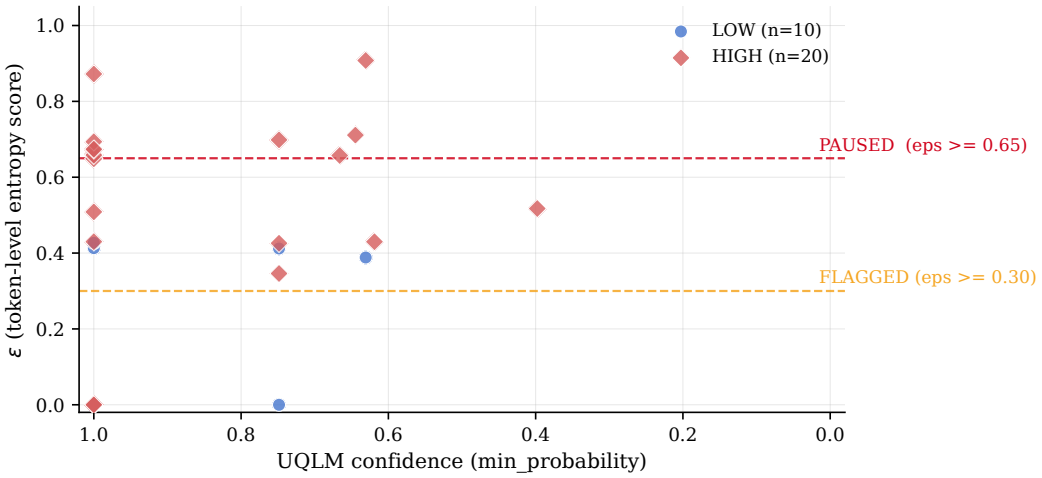

Figure 6: UQLM returned confidence ≥ 0.62 on 29/30 prompts (including 19/20 HIGH). Its sole detection is pandas `concat`, where syntax genuinely varied across samples; ε separates the two populations by construction.

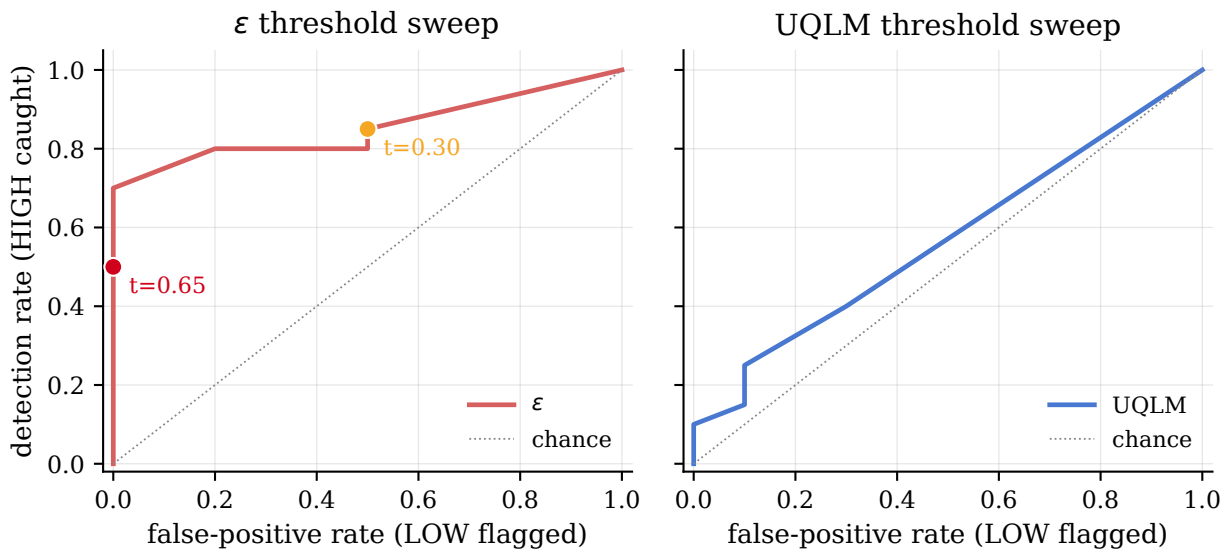

Figure 7: Sweeping the decision threshold for each method on GPT-4o. ε has a wide band where HIGH detection dominates LOW FP; UQLM's useful operating range collapses as threshold rises.

5.2 Single-token log-probability baselines

`min_logprob` flags a function if any selected token's log-probability falls below a threshold — same raw signal, top-1 only.

Method	HIGH det. @ 30% LOW FP	HIGH det. @ 50% LOW FP
`min_logprob`	45–55%	55–65%
ε(max, AST-filtered)	**70**%	**85**%

Table 7: Matched LOW FP budgets across three models. Gap is at near-tie tokens (e.g., 0.48/0.42): `min_logprob` sees 0.48 and does not fire; εsees near-uniform top-k entropy and does.

A low top-1 probability does not distinguish a cosmetic name from an API selector from a whitespace artifact. AST filtering plus top-k entropy closes the gap: the signal is in the raw logprobs, inaccessible without filtering.

5.3 Static analysis, linters, and model coverage

Static analysis catches *known*-bad patterns for libraries with rules written; εfires where the model itself hesitated, including decisions not yet in any ruleset (a library updated last week, a half-migrated codebase). Static analysis is the floor under ε. Primary experiments use OpenAI plus DeepSeek V3 via Together AI; Anthropic exposes logprobs via streaming, vLLM via full top-k. The method requires only top-k; no model-internal access.

6 Discussion

6.1 Limitations

Confident wrongness. εcannot detect a model that is wrong and confident. If training placed all mass on a deprecated API, the token probability is sharp and $\varepsilon$is low; the system emits the wrong code with COMPLETE status. Fundamental limit of any generation-time signal; motivates static-analysis integration.

Confidently-resolved domains. Prompts where every model has converged on a strong training-data prior produce $\varepsilon = 0.000$ regardless of correctness. Current-UTC datetime retrieval and HTTP-GET-to-JSON fall here across all three models. For stable widely-deployed APIs this is desirable; for deprecated-but-confidently-recalled APIs it is Scenario B's failure mode.

Small-sample calibration. Main calibration is 30 prompts per model; Wilson intervals on HIGH detection are $\pm$17pp at $n = 20$. We claim recall-first design plus an empirical miss rate of zero on the sets evaluated, not a proof of perfect recall.

Type 3 floor. Implementation-approach uncertainty on simple prompts (brute vs. hash two-sum) legitimately raises εon correct code. AST filtering does not remove it; the $\sim 50\%$ LOW FP is predominantly Type 3 and unlikely to shrink without sacrificing recall.

LLM-judged ground truth. HIGH ground truth comes from the LLM review loop, not human verification. Circularity exists where the judged model shares a family with the judge (GPT-4o-mini judged by GPT-4o); we state it rather than hide it.

Hyperparameters. The cascaded tier algorithm (six checks, three thresholds) was calibrated on GPT-4o; other models inherit these. The binary FLAGGED+ gate is robust; internal tier placement is a refinement on top.

6.2 The rising effective cost of AI error

The cost of an individual AI coding error is rising: more code per developer-hour, less scrutiny per line, compound pipelines amplifying upstream errors. A parallel reviewer architecture decouples review latency from developer attention: in our evaluation, batched reviews completed in seconds at fractions of a cent per entry.

6.3 Context learning as a microcosm

Configuration 2B injects learned FP patterns into the reviewer's context on subsequent runs: clearance rose $53.2\% \rightarrow 68.7\%$, late FPs fell $4 \rightarrow 0$. This is in-context improvement on a fixed model; training-time improvement would be expected to compound with it.

6.4 Future work

Pipeline propagation: upstream εshould propagate as prior uncertainty into downstream generations, modulating sampling temperature or review threshold. **Open-source models:** vLLM exposes full top-k; a Llama/Qwen/Mistral calibration run would test whether the recall-first boundary holds outside OpenAI. **Per-token intervention:** surface the high-εtoken as an inline editor marker, or

trigger targeted re-decoding with lower temperature on just the uncertain span. **Cross-decision consistency:** Scenario D's mismatches argue for AST-level consistency checks over ε(e.g., `async def` + blocking HTTP client).

7 Conclusion

We introduced ε, a per-token normalized Shannon entropy over top-k log-probabilities, AST-filtered and max-aggregated into a single score per generated function. $\varepsilon$is a filter, not a detector: it achieves the recall target established in §4.5. The contribution is the two-stage system. Pairing a recall-maximizing filter with a parallel LLM reviewer produces an architecture in which nothing uncertain ships unreviewed and the reviewer scales with generation speed.

The framing matters. Multi-sample uncertainty methods produce almost no signal on code generation because code uncertainty is intra-generational — visible in the single-generation token distribution, invisible across re-samples. Once the distinction is named, the design of εfollows directly. The same mechanism — early-token commitment forcing later tokens — plausibly applies in other structured-output domains (SQL, configuration files, typed program synthesis); empirical verification is left to future work. Generation is fast; review is the bottleneck. A filter that misses nothing plus a parallel reviewer moves review off the developer's critical path.

References

[1] Anastasios N Angelopoulos, Stephen Bates, Adam Fisch, Lihua Lei, and Tal Schuster. Conformal risk control. In *International Conference on Learning Representations (ICLR)*, 2024.

[2] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. Program synthesis with large language models, 2021.

[3] Dylan Bouchard, Mohit Singh Chauhan, David Skarbrevik, Ho-Kyeong Ra, Viren Bajaj, and Zeya Ahmad. UQLM: A python package for uncertainty quantification in large language models. *Journal of Machine Learning Research*, 2026. arXiv:2507.06196.

[4] Bei Chen, Fengji Zhang, Anh Nguyen, Daoguang Zan, Zeqi Lin, Jian-Guang Lou, and Weizhu Chen. CodeT: Code generation with generated tests, 2022.

[5] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code, 2021.

[6] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024.

[7] Alain Jungo, Raphael Meier, David Alleysson, and Mauricio Reyes. Analyzing the quality and challenges of uncertainty estimations for brain tumor segmentation with deep neural networks. *Frontiers in Neuroscience*, 2020.

[8] Saurav Kadavath, Tom Conerly, Amanda Askell, Tom Henighan, Dawn Drain, Ethan Perez, Nicholas Schiefer, Zac Hatfield-Dodds, Nova DasSarma, Eli Tran-Johnson, et al. Language models (mostly) know what they know, 2022.

[9] Lorenz Kuhn, Yarin Gal, and Sebastian Farquhar. Semantic uncertainty: Linguistic invariances for uncertainty estimation in natural language generation. In *International Conference on Learning Representations (ICLR)*, 2023.

[10] Victor Quach, Adam Fisch, Tal Schuster, Adam Yala, Jae Ho Sohn, Tommi S Jaakkola, and Regina Barzilay. Conformal language modeling. In *International Conference on Learning Representations (ICLR)*, 2024.

[11] Neeraj Varshney, Wenlin Yao, Hongming Zhang, Jianshu Chen, and Dong Yu. A stitch in time saves nine: Detecting and mitigating hallucinations of LLMs by validating low-confidence generation. In *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics (EACL)*, 2024.

[12] He Ye, Matias Martinez, and Martin Monperrus. SelfAPR: Self-supervised program repair with test execution diagnostics. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2022.